

2. Масштабирование баз данных

2.1 Сервис ставок

Цель: держать P95 $\leq$ 80–100 мс при высоких RPS.

- Сервис не имеет собственной БД, используется Read replica PostgreSQL сервиса рекламных кампаний.
- Redis: репликация по умолчанию (MasterReplica) для отказоустойчивости. Для сверхвысокой доступности можно использовать Redis Cluster.

CQRS

Частичный CQRS: поток на запись (идёт через Kafka. Чтение идёт напрямую из Redis. Это снимает нагрузку с мастер и позволяет обновлять лимиты без блокировок.

2.2 Сервис рекламных кампаний

Цель: целостность данных, поддержка сложных фильтров, умеренная нагрузка на запись.

Репликация и read replicas

- PostgreSQL MasterSlave: 1 мастер + read replica.
- Writes идут только на мастер; reads (списки кампаний, фильтры по гео/категориям) — на реплики.

Шардирование

На начальном этапе не требуется.

При росте до значительного числа кампаний (например, более 1 м, в зависимости от метрик производительности): шардировать (например) таблицу рекламных кампаний по account_id (или dsp_id), чтобы кампании одного рекламодателя лежали на одном шарде. Это упрощает права доступа и отчёты по аккаунту.

CQRS

Не обязателен для базовой CRUDлогики.

2.3 Сервис статистики (Stats Service)

Цель: высокая пропускная способность на запись, быстрые агрегации по времени и атрибутам.

Репликация

ClickHouse: нативная репликация между нодами. Данные автоматически реплицируются по ZooKeeper/ClickHouse Keeper.

Шардирование

- Предлагается шардировать данные (таблицу) события показов/кликов по ключу времени `event_time` (партиционирование по дням) а также `ad_unit_id/campaign_id`.
- Типовой паттерн: партиции по дням, шарды по диапазонам `ad_unit_id`. Это ускоряет вставку и позволяет делать быстрые выборки по временным окнам.

CQRS

Встроенный CQRS в ClickHouse: поток событий (raw) → агрегации (AggregatedMergeTree, SummingMergeTree) → витрины.

Raw - слой хранит все события; агрегированные таблицы — готовые срезы для API и дашбордов.

2.4 Сервис аналитики (Analytics Service)

Цель: сложные аналитические запросы, исторические данные, поддержка Визуальных инструментов.

Репликация

ClickHouse: нативная репликация между нодами. Данные автоматически реплицируются по ZooKeeper/ClickHouse Keeper.

Шардирование

Аналогично сервису статистики: шардировать данные (таблицу) по `event_date` и шардирование по `campaign_id` либо по диапазонам ID.

Витрины (агрегации) могут иметь отдельную схему шардирования под частые запросы отчетов.

Read replicas

Read replicas использовать для внешних (если будут) аналитических сервисов (Superset, PowerBI) делают много параллельных запросов. Реплики позволяют распределить эту нагрузку и не мешать потоковой записи.

- Можно выделить отдельный кластер или пул реплик только под BI -запросы.

CQRS

Встроенный CQRS в ClickHouse: поток событий (raw) → агрегации (AggregatedMergeTree, SummingMergeTree) → витрины.

Raw - слой хранит все события; агрегированные таблицы — готовые срезы для API и дашбордов.

2.5 Сервис финансов (Billing/Finance Service)

Цель: максимальная надёжность, строгая консистентность, аудит, соответствие требованиям учёта.

Репликация и read replicas

PostgreSQL MasterSlave (полусинхронная): 1 мастер, 2–3 реплики.

Все финансовые проводки (начисления, списания, корректировки) обращаются строго на мастер.

Read replicas используются для отчётов и сверки, чтобы не нагружать мастер.

Шардирование

На начальном этапе не требуется.

Если объём транзакций станет очень большим: шардировать по account_id + event_date. Это сохраняет целостность операций внутри аккаунта и упрощает архивацию старых периодов.

CQRS

Поток на запись напрямую PostgreSQL (в транзакции).

Поток чтения: отдельные таблицы/витрины для отчётов (сальдо, акты, сверки).

Такой подход позволяет отделить критичные проводки от тяжёлых отчётных запросов и обеспечить аудит всех изменений.